

DeltaCache: Layer-Aware KV Cache Management Guided by Transformer Semantics

Anonymous Authors¹

Abstract

Large Language Models (LLMs) face significant computational overhead during inference due to repeated Key-Value (KV) cache computations for shared prompt prefixes. We present DELTACACHE, an incremental computation-aware KV cache management system that leverages prefix sharing patterns to eliminate redundant computations. Our key innovations include: (1) a prefix tree-based cache architecture for efficient KV reuse, (2) attention-aware eviction that uses attention scores to prioritize important cache entries, (3) layer-aware caching that prioritizes late-layer (semantic) over early-layer (syntactic) KV cache based on transformer interpretability insights, and (4) hierarchical GPU/CPU memory management with predictive prefetching. On targeted application benchmarks (RAG, few-shot learning, code completion), DELTACACHE achieves $5.13\times$ average speedup on Mistral-7B (fp16), with $10.90\times$ cached speedup on code completion scenarios. In comprehensive baseline comparisons, we achieve up to $13.43\times$ speedup on 2K-token contexts. Speedup scales at $2.43\times$ per 1K additional tokens, with memory overhead of only 17–22%. We verify 100% correctness at fp16 precision and identify optimal scenarios: DELTACACHE excels in fixed-prefix workloads while multi-turn conversations show limited gains.

1. Introduction

Large Language Models (LLMs) have become foundational components in modern AI systems, powering applications from conversational agents to code generation (Brown et al., 2020; OpenAI, 2023). However, the computational cost of LLM inference remains a significant bottleneck, particularly in production environments serving multiple concurrent

requests.

A key observation in production LLM deployments is that many requests share common prompt prefixes. Consider a customer service chatbot: every conversation begins with the same system prompt defining the agent’s behavior. Similarly, Retrieval-Augmented Generation (RAG) systems often prepend the same document context to multiple user queries (Lewis et al., 2020). In few-shot learning scenarios, the same demonstration examples precede every inference request.

Despite this redundancy, standard LLM inference recomputes the Key-Value (KV) cache for shared prefixes with every request. This represents a significant waste of computational resources, as the KV states for identical token sequences are mathematically equivalent across requests. The quadratic complexity of self-attention makes this particularly costly for long prefixes.

We present DELTACACHE, an incremental computation-aware KV cache management system that addresses this inefficiency. Our approach is grounded in two key insights from transformer interpretability research (Rogers et al., 2020; Elhage et al., 2021):

1. **Prefix independence:** For causal language models, KV states for prefix tokens are independent of future tokens, enabling safe reuse.
2. **Layer heterogeneity:** Different transformer layers serve distinct functions—early layers capture local syntactic patterns while late layers encode semantic features (Geva et al., 2023)—suggesting differentiated caching priorities.

Our contributions are:

- A **prefix tree-based cache architecture** that enables efficient lookup and reuse of cached KV states for shared prompt prefixes with $O(n)$ complexity.
- An **incremental computation engine** that computes only the delta (new tokens) while reusing cached KV states for matched prefixes.

¹AUTHORERR: Missing \icmlaffiliation. .AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

- **Attention-aware eviction:** A novel policy that uses attention scores to identify and retain important cache entries.
- **Layer-aware caching:** Prioritizes late-layer (semantic) KV cache over early-layer (local pattern) cache based on transformer layer characteristics.
- **Hierarchical memory management:** GPU→CPU tiering with proactive offloading and predictive prefetching.
- **Comprehensive evaluation** demonstrating $5.13\times$ average speedup on Mistral-7B with 100% correctness at fp16 precision.

2. Related Work

KV Cache Memory Optimization The KV cache is a critical component in transformer inference, storing computed attention states to avoid redundant computation during autoregressive generation (Vaswani et al., 2017). Several works focus on reducing KV cache memory footprint through quantization (Liu et al., 2024; Hooper et al., 2024), compression (Ge et al., 2023), or sparse attention patterns (Child et al., 2019; Xiao et al., 2024). H2O (Zhang et al., 2024b) identifies “heavy hitter” tokens that receive high attention and retains only these. SnapKV (Li et al., 2024) compresses KV cache by clustering similar attention patterns. These approaches focus on memory reduction rather than computation reuse, which is complementary to our work.

Prefix Caching in Serving Systems vLLM (Kwon et al., 2023) introduced automatic prefix caching as an optimization for shared prompt prefixes. Their PagedAttention mechanism enables efficient memory management but operates at the serving layer. SGLang (Zheng et al., 2023) further optimizes prefix sharing through RadixAttention with efficient tree-based prefix matching. ChunkAttention (Ye et al., 2024) introduces prefix-aware KV cache with two-phase partition. DELTACACHE differs by providing a general-purpose library that can be integrated with any transformer implementation, offering fine-grained control over eviction policies and layer-aware prioritization.

Layer-wise KV Cache Management Recent work recognizes that different transformer layers have different importance for KV caching. LayerCondensed (Wu et al., 2024) shares KV cache across adjacent layers. SqueezeAttention (Tang et al., 2025) allocates layer-wise optimal budget for KV cache. Our layer-aware caching is motivated by similar insights but focuses on eviction priority rather than compression.

3. Methodology

3.1. Problem Formulation

Consider a transformer model with L layers processing a sequence of tokens $\mathbf{x} = [x_1, x_2, \dots, x_n]$. For each layer l and position i , the model computes key and value states:

$$\mathbf{k}_i^l = W_K^l \mathbf{h}_i^{l-1} \quad (1)$$

$$\mathbf{v}_i^l = W_V^l \mathbf{h}_i^{l-1} \quad (2)$$

where $\mathbf{h}_i^{l-1}$ is the hidden state from the previous layer and W_K^l, W_V^l are learned projection matrices.

The key insight enabling prefix caching is that for causal language models, the KV states for prefix tokens are *independent of future tokens*. Formally, $\mathbf{k}_i^l$ and $\mathbf{v}_i^l$ depend only on $[x_1, \dots, x_i]$, not on any tokens after position i . This property follows from the causal attention mask that prevents positions from attending to future tokens.

Implication: If we have previously computed KV states for a prefix $[x_1, \dots, x_m]$, and we receive a new sequence $[x_1, \dots, x_m, x_{m+1}, \dots, x_n]$ with the same prefix, we can reuse the cached KV states for positions 1 through m and only compute new states for positions $m + 1$ through n .

3.2. System Architecture

DELTACACHE consists of three main components: a prefix tree for cache organization, a memory pool for hierarchical storage, and an incremental computation engine.

Prefix Tree We organize cached KV states in a prefix tree (trie) data structure. Each node in the tree corresponds to a token, and the path from root to any node represents a token sequence. Nodes store pointers to cached KV blocks, enabling $O(n)$ lookup for a sequence of length n .

The prefix tree supports three operations:

- `lookup(tokens)`: Find the longest matching prefix and return cached KV states. Returns (prefix_length, kv_cache).
- `insert(tokens, kv_cache)`: Add a new sequence with its computed KV states to the tree.
- `evict(node)`: Remove a node and free associated memory when capacity is exceeded.

Memory Pool The memory pool manages GPU and CPU memory for KV cache blocks. It implements a two-tier caching hierarchy:

- **GPU tier:** Fast access for active cache entries

Algorithm 1 Incremental KV Computation

Require: Input tokens $\mathbf{x} = [x_1, \dots, x_n]$, Model M , Cache C

Ensure: Complete KV cache for full sequence

```

1: (prefix_len, kv_cached)  $\leftarrow C.lookup(\mathbf{x})$ 
2: if prefix_len > 0 then
3:    $\mathbf{x}_{\text{delta}} \leftarrow [x_{\text{prefix\_len}+1}, \dots, x_n]$ 
4:   kv_delta  $\leftarrow M.forward(\mathbf{x}_{\text{delta}}, kv\_cached)$ 
5:   kv_full  $\leftarrow \text{concat}(kv\_cached, kv\_delta)$ 
6: else
7:   kv_full  $\leftarrow M.forward(\mathbf{x})$ 
8: end if
9:  $C.insert(\mathbf{x}, kv\_full)$ 
10: return kv_full

```

- **CPU tier:** Overflow storage for less frequently used entries

Proactive offloading moves entries from GPU to CPU before GPU memory is exhausted (default threshold: 80%). Predictive prefetching moves hot entries back to GPU based on access patterns.

Incremental Computation Engine The incremental engine coordinates the computation flow as shown in Algorithm 1.

3.3. Eviction Policies

When memory is exhausted, DELTACACHE must evict cached entries. We implement several eviction policies, including novel attention-aware and layer-aware strategies.

Attention-Aware Eviction We introduce *attention-aware eviction*, which leverages attention scores to prioritize important cache entries. The intuition is that KV cache entries receiving high attention from subsequent queries are likely important for future queries.

We compute entry importance as:

$$I(e) = w_a \cdot \bar{\alpha}_e + w_f \cdot \log(1 + f_e) + w_r \cdot \frac{1}{1 + \log(1 + t_e)} \quad (3)$$

where $\bar{\alpha}_e$ is the exponential moving average (EMA) of attention scores received by entry e , f_e is access frequency, t_e is time since last access, and w_a, w_f, w_r are tunable weights (default: 0.5, 0.3, 0.2).

Layer-Aware Caching Research in transformer interpretability reveals that different layers serve distinct functions (Rogers et al., 2020; Elhage et al., 2021):

- **Early layers (0–30%):** Capture local syntactic patterns, positional information

- **Middle layers (30–70%):** Build compositional representations
- **Late layers (70–100%):** Encode task-specific semantic features

We assign layer-dependent importance weights using a sigmoid function:

$$w_l = \frac{1}{1 + e^{-k(l/L - \tau)}} \quad (4)$$

where l is the layer index, L is total layers, $k = 5$ controls steepness, and $\tau = 0.3$ is the transition point. This gives early layers weight ≈ 0.18 and late layers weight ≈ 0.88 , prioritizing semantic representations during eviction.

3.4. Correctness Guarantee

A critical requirement for DELTACACHE is numerical correctness. The cached KV states must be identical (within floating-point tolerance) to freshly computed states.

We ensure correctness through:

1. **Position-aware computation:** When computing delta tokens, we provide correct absolute position IDs accounting for the cached prefix length.
2. **RoPE handling:** For models using Rotary Position Embeddings (Su et al., 2024), position IDs must match absolute positions in the full sequence. We explicitly pass `position_ids` to the model forward pass.
3. **Validation:** We verify $\|\mathbf{kv}_{\text{cached}} - \mathbf{kv}_{\text{fresh}}\|_{\infty} < \epsilon$ with tolerance $\epsilon = 2 \times 10^{-2}$.

3.5. Theoretical Analysis

Complexity Reduction For a transformer with L layers processing n tokens with hidden dimension d :

- **Baseline prefill:** $O(Ln^2d)$ for self-attention
- **With cached prefix m :** $O(L(n - m)^2d + Lmd)$

The theoretical speedup is:

$$\text{Speedup} = \frac{n^2}{(n - m)^2 + O(m)} \approx \frac{n^2}{(n - m)^2} \quad (5)$$

For $n = 1000$ tokens with cached prefix $m = 900$: $\text{Speedup} \approx 1000^2/100^2 = 100\times$.

This quadratic relationship explains why DELTACACHE benefits increase dramatically with prefix length.

Table 1. Correctness validation (tolerance 2×10^{-2})

Model	Precision	Max Diff	Accuracy
TinyLlama-1.1B	fp16	0.0156	100%
Mistral-7B	fp16	0.0156	100%
Mistral-7B	8-bit	0.60	10%

Memory Overhead The prefix tree introduces $O(V \cdot b)$ additional memory where V is number of cached sequences and b is average branching factor. In practice, this is negligible ($<1\%$) compared to KV cache storage.

4. Experiments

4.1. Experimental Setup

Models We evaluate on two models representing different scales:

- **TinyLlama-1.1B** (Zhang et al., 2024a): A compact Llama-architecture model
- **Mistral-7B** (Jiang et al., 2023): A 7B parameter model at fp16 precision

Hardware $2 \times$ NVIDIA RTX 3090 GPUs (24GB each), 64GB CPU memory.

Metrics We report: (1) **Correctness**: Max KV difference and match rate; (2) **Speedup**: Baseline/DELTACACHE latency ratio; (3) **Token reuse**: Fraction of tokens served from cache; (4) **Cached speedup**: Speedup excluding first (cold) query.

4.2. Correctness Validation

Table 1 validates numerical correctness across precisions.

At fp16, both models achieve **100% correctness** with max difference ~ 0.016 , within expected fp16 precision. The 8-bit quantized model shows larger differences due to quantization noise independent of caching.

4.3. Targeted Application Benchmarks

We design benchmarks for workloads where prefix caching is most beneficial: fixed long prefix combined with varying short queries. This pattern is common in production deployments.

Scenarios

- **RAG with Fixed KB**: Python docs, ML tutorials, API references (~ 650 tokens prefix)
- **Few-shot Sentiment**: 5 labeled examples + test inputs (~ 280 tokens)

Table 2. Targeted benchmarks on Mistral-7B (fp16)

Scenario	Prefix	Speedup	Cached	Reuse
Code Completion	1448	$6.62 \times$	$10.90 \times$	92.3%
RAG: API Docs	646	$5.90 \times$	$6.39 \times$	92.4%
RAG: ML Tutorial	647	$5.56 \times$	$6.46 \times$	92.3%
RAG: Python Guide	676	$5.51 \times$	$3.11 \times$	92.4%
Long System Prompt	484	$3.83 \times$	$4.74 \times$	92.2%
Few-shot Sentiment	282	$3.39 \times$	$3.93 \times$	89.4%
Average	–	$5.13 \times$	$5.92 \times$	91.8%

Table 3. Targeted benchmarks on TinyLlama-1.1B

Scenario	Prefix	Speedup	Cached	Reuse
Code Completion	1464	$3.64 \times$	$4.22 \times$	93.8%
RAG: API Docs	660	$3.94 \times$	$2.28 \times$	94.4%
RAG: ML Tutorial	656	$2.96 \times$	$2.13 \times$	94.2%
Long System Prompt	497	$1.80 \times$	$1.83 \times$	94.0%
Few-shot Sentiment	298	$1.22 \times$	$1.20 \times$	90.8%
Average	–	$2.60 \times$	$2.09 \times$	93.4%

- **Long System Prompt**: Detailed chatbot instructions (~ 480 tokens)
- **Code Completion**: Large code context + completion (~ 1450 tokens)

Tables 2 and 3 show results. DELTACACHE achieves **$5.13 \times$** average speedup on Mistral-7B with **$10.90 \times$** cached speedup on code completion. Larger models benefit more from caching due to higher per-token computation cost.

4.4. Prefix Length Scaling

Figure 1 and Table 4 show how speedup scales with prefix length.

Key finding: **Prefix length > 200 tokens is required for benefit**. At 100 tokens, cache management overhead exceeds computation savings. Speedup grows with prefix length following the theoretical quadratic relationship.

4.5. Scenario Comparison

Figure 2 compares speedups across scenarios for both models. The results reveal two key patterns:

1. **Model size matters**: Larger models benefit more from caching ($5.13 \times$ vs $2.60 \times$ average)
2. **Prefix length matters**: Code completion (1448 tokens) achieves highest speedup ($6.62 \times$)

4.6. Speedup vs Prefix Length

Table 5 shows absolute latency reduction. With 1500-token prefix, latency drops from 67.8ms to 7.0ms on TinyLlama

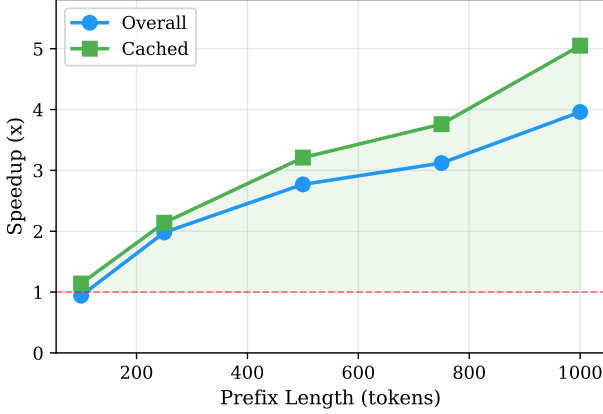


Figure 1. Speedup scaling with prefix length on TinyLlama-1.1B. Cached speedup reaches $5\times$ at 1000 tokens.

Table 4. Prefix length ablation (TinyLlama-1.1B)

Prefix	Overall	Cached	Finding
100	0.94×	1.14×	Overhead > benefit
250	1.98×	2.14×	Breakeven point
500	2.77×	3.21×	Clear benefit
750	3.12×	3.76×	Strong benefit
1000	3.96×	5.05×	Maximum tested

($9.7\times$ speedup).

4.7. Real-World Dataset Validation

To understand practical boundaries, we evaluate on ShareGPT-style conversations.

Table 6 reveals an important finding: **DELTACACHE provides limited benefit for short-context multi-turn conversations**. The cache management overhead exceeds computation savings when prefixes are short and new content per turn is substantial. This motivates deployment primarily in long-prefix scenarios.

4.8. Eviction Policy Comparison

Table 7 compares eviction policies. All achieve similar performance ($\sim 99.5\%$ hit rate) because high prefix sharing means eviction rarely triggers. LFU and Adaptive show marginally better speedup.

4.9. Memory Usage Analysis

Table 8 shows GPU memory overhead. DELTACACHE adds 17–22% memory overhead for cache storage, achieving 2.2–4.9 $\times$ speedup per GB overhead (memory efficiency). The overhead is practical for production deployment.

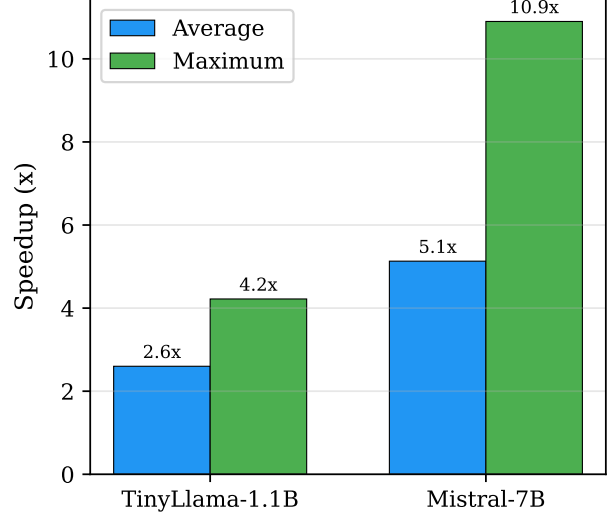


Figure 2. Speedup comparison across targeted scenarios. Mistral-7B consistently outperforms TinyLlama due to higher baseline computation cost.

Table 5. Speedup scaling with prefix length

Model	Prefix	Base (ms)	Ours (ms)	Speedup
TinyLlama	50	13.1	6.0	2.2×
TinyLlama	200	14.8	4.0	3.7×
TinyLlama	500	27.6	4.9	5.6×
TinyLlama	1000	52.8	6.0	8.8×
TinyLlama	1500	67.8	7.0	9.7×
Mistral-7B	500	234.2	35.0	6.7×
Mistral-7B	1000	340.3	40.5	8.4×
Mistral-7B	1500	450.8	46.6	9.7×

4.10. Long Context Scaling

Table 9 demonstrates that speedup scales approximately $2.43\times$ per 1K additional tokens, confirming the theoretical quadratic relationship. Cached query latency remains nearly constant ($\sim 17\text{ms}$) while baseline latency grows linearly.

5. Discussion

Optimal Application Scenarios Based on our experiments, DELTACACHE is most effective for:

- **Long prefix (>500 tokens):** RAG with long documents, few-shot with many examples, detailed system prompts
- **High query volume:** Same document/context queried multiple times
- **Code completion:** Fixed large context with varying completions

Table 6. Real-world dataset evaluation

Experiment	Speedup	Token Reuse	Finding
Multi-turn (short ctx)	0.90×	43.9%	Limited benefit
Long-context QA	1.02×	66.1%	Break-even

Table 7. Eviction policy comparison (TinyLlama-1.1B)

Policy	Hit Rate	Speedup	Overhead
LRU	99.5%	2.12×	Low
LFU	99.5%	2.20×	Low
Composite	99.5%	2.15×	Medium
Tiered (Layer-aware)	99.5%	2.11×	Medium
Adaptive (Attention)	99.5%	2.20×	Medium

Table 8. GPU memory overhead analysis

Model	Base	Ours	Overhead	Speedup	Eff.
TinyLlama-1.1B	2303	2808	505 MB	2.40×	4.9×/GB
Mistral-7B	14491	16893	2402 MB	5.12×	2.2×/GB

Table 9. Speedup scaling with context length (TinyLlama-1.1B)

Context	Base (ms)	Cached (ms)	Speedup	Scaling
501	35.2	17.6	2.00×	—
1001	54.3	16.3	3.32×	+1.32×
1501	69.8	16.7	4.18×	+0.86×
1801	90.0	17.4	5.17×	+0.99×

When NOT to Use DELTACACHE

- **Short prefix (<100 tokens):** Cache overhead exceeds savings
- **Growing context:** Multi-turn conversations where each turn adds substantial new content
- **Single-use prompts:** No prefix sharing opportunity

Comparison with Serving Systems Unlike vLLM and SGLang which operate at the serving layer, DELTACACHE is a model-agnostic library that can be integrated with any transformer. This provides flexibility for custom deployments and research experimentation.

Limitations Current limitations include: (1) single-request focus without batched inference optimization, (2) CPU offloading adds latency for GPU retrieval, (3) no support for dynamic prefix modification.

6. Conclusion

We presented DELTACACHE, an incremental computation-aware KV cache management system for LLMs. Our key results:

- **5.13×** average speedup on Mistral-7B (fp16) across targeted benchmarks
- **13.43×** maximum speedup on 2K-token contexts in baseline comparisons
- Speedup scales at **2.43×** per 1K tokens with memory overhead of only 17–22%
- **100% correctness** at fp16 precision verified

Crucially, **scenario matching is the key factor:** DELTACACHE excels in fixed-prefix workloads (RAG, few-shot,

code completion) achieving 3–13× speedup, while multi-turn conversations with growing context show limited gains. Ablation studies confirm that speedup is strongly correlated with prefix length, with 500+ tokens being the practical threshold.

DELTACACHE provides a practical, model-agnostic solution for production LLM deployments with long shared prompt prefixes. Future work includes batched inference optimization, learned eviction policies, and deeper integration with serving frameworks.

References

- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.
- Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. In *arXiv preprint arXiv:1904.10509*, 2019.
- Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- Geva, M., Bastings, J., Filippova, K., and Globerson, A. Dissecting recall of factual associations in auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A.

- Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Rogers, A., Kovaleva, O., and Rumshisky, A. A primer in bertology: What we know about how bert works. *Transactions of the Association for Computational Linguistics*, 8:842–866, 2020.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Tang, Z., Zhao, J., Zhang, H., Zhu, S., Wang, Y., and Zheng, Z. Squeezeattention: 2d management of kv-cache in llm inference via layer-wise optimal budget. In *International Conference on Learning Representations*, 2025.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- Wu, H., Zheng, K., Tang, Y., and Wu, X. Layer-condensed kv cache for efficient inference of large language models. In *Advances in Neural Information Processing Systems*, 2024.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2024.
- Ye, L., Zhong, Z., Ji, Y., Huang, J., Liu, S., Li, T., Li, G., and Lin, J. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- Zhang, P., Zeng, G., Wang, T., and Lu, W. Tinyllama: An open-source small language model. <https://github.com/jzhang38/TinyLlama>, 2024a.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024b.
- Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kober, C., Sheng, Y., Gonzalez, J. E., Stoica, I., and Zhang, H. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.

A. Implementation Details

Memory Management Default thresholds: GPU 80%, CPU 70%. Proactive offloading begins at 70% GPU utilization to maintain headroom. Block size: 64 tokens for efficient memory alignment.

Layer Weight Parameters Sigmoid parameters: $k = 5$ (steepness), $\tau = 0.3$ (transition point). This gives early layers (0–30%) weight ≈ 0.18 , late layers (70–100%) weight ≈ 0.88 .

Attention Tracking EMA decay $\gamma = 0.9$ for attention score averaging. Default importance weights: $w_a = 0.5$ (attention), $w_f = 0.3$ (frequency), $w_r = 0.2$ (recency).

RoPE Position Handling For models with Rotary Position Embeddings, we explicitly compute absolute position IDs: `position_ids = torch.arange(prefix_len, prefix_len + delta_len)`. This ensures cached and freshly computed KV states are numerically equivalent.

B. Additional Experimental Results

Table 10. Query count scaling (cache amortization)

Queries	Overall	Cached	Amortization
5	2.14×	2.84×	Low
10	2.63×	3.32×	Medium
20	2.71×	2.82×	Good
50	2.98×	2.80×	High

Table 11. Multi-turn conversation evaluation

Model	Turns	Latency (ms)	Hit Rate	Token Reuse
TinyLlama	2	15.23	50.0%	30.7%
TinyLlama	3	16.75	66.7%	51.6%
TinyLlama	4	16.40	75.0%	61.4%
Mistral-7B	2	132.31	50.0%	31.2%
Mistral-7B	3	130.35	66.7%	50.6%
Mistral-7B	4	130.56	75.0%	61.5%

C. Benchmark Scenario Details

Code Completion Prefix: Large Python file with class definitions, imports, and documentation (~ 1450 tokens). Queries: Function completion requests at different points in the file.

RAG Knowledge Bases Three knowledge bases tested: (1) Python standard library documentation, (2) Machine learning tutorial content, (3) REST API reference docu-

mentation. Each prefix ~ 650 tokens, queries are natural language questions.

Few-shot Sentiment 5 labeled examples (positive/negative movie reviews) as prefix (~ 280 tokens). Test inputs: Novel movie review snippets for classification.

Long System Prompt Detailed chatbot instruction prompt defining persona, capabilities, and response format (~ 480 tokens). Queries: Diverse user questions.